claude-windows / 166d7059-d8ac-4fdd-a8c0-072cdaa7eccd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\166d7059-d8ac-4fdd-a8c0-072cdaa7eccd\subagents\workflows\wf_66c34d9d-057\agent-adba0f3d86d5b1d99.jsonl`
- SHA-256: `260820cdcef434648639e1417bdd966b74970d59458a5b2d9314ecfacb41fe67`
- Source modified: `2026-06-18T05:33:49+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_66c34d9d-057`
- session_id: `166d7059-d8ac-4fdd-a8c0-072cdaa7eccd`

Transcript

1. user (2026-06-18T05:31:43.303Z)

```
WORKTREE (cd here; code is origin/master f5f339a): C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13
Run python as:  cd "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13" && PYTHONPATH=. python <your_script>
GOLDEN/manifest/features dir (ABSOLUTE ? pass this, the worktree output/ is empty): C:/Users/avidu/Projects/badminton-highlight-indexer/output

REPRODUCE RECIPE (use the public API; write your own short script to scratch/lens_<name>.py):
  import os
  from dataclasses import replace
  from backend.eval import rally_seg_eval as rse, eval_stats
  from backend.eval.windowing import SERVED
  from backend.eval.tolerance_metrics import over_seg_guard
  OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"
  golden = [g for g in rse.load_golden(os.path.join(OUT,"human_lovo_manifest.json"), OUT)
            if g["gts"] and os.path.exists(g["trajectory"])]   # expect 13
  baseline  = SERVED                                          # all windowing knobs OFF
  cap12     = replace(SERVED, max_window_duration=12.0)
  cap6      = replace(SERVED, max_window_duration=6.0)
  milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                      inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
  rows = lambda p: {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}  # each row: tol_f1,
      # f1, merge_rate, split_rate, merge_count, split_count, num_pred, num_gt, name, dend_abs_median
  # paired sign-test on a metric: eval_stats.paired_delta_ci([base[n][k] for n in names],[cand[n][k] for n in names])
  #   returns {mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test:{wins,losses,ties,p_value}}
  # over-seg guard: over_seg_guard(list(base_rows.values()), list(cand_rows.values()))
  #   returns {passes, strict_passes, base_net, cand_net, net_delta, no_merge_rate_regress, merge_rate_regress:[...]}
  # NOTE: rse.evaluate may print "R5 blob-fallback: forced ..." lines for the blob clip ? that is the
  #   R5 logger and is HARMLESS/irrelevant here (the milestone does NOT enable blob_fallback). Ignore it.

THE CLAIM TO FALSIFY: "The R1 milestone (W1 cap=6 + R4 inactivity-end=1.5/rally_thresh=0.20/min_density=0.30, no W2, no R5) still clears the net-over-seg promotion bar at n=13 ? a SIGNIFICANT
R2 tolF1 win over baseline AND no honest over-seg regression ? so config-flip PR #204 still stands."

MY (the orchestrator's) measured n=13 numbers, for you to INDEPENDENTLY confirm or REFUTE ? do
```

```
NOT
trust them, recompute:
  - baseline->milestone tolF1: ?+0.2222 [+0.1522,+0.2859], sign 12/0/1, p=0.0005
  - cap12->cap6 tolF1:         ?+0.1099 [+0.0703,+0.1526], sign 12/0/1, p=0.0005
  - cap6->+R4 tolF1:           ?+0.0397 [+0.0145,+0.0651], sign 9/1/3,  p=0.0215   (ONE loss ?
find it)
  - mean merge_rate baseline->milestone: 0.651 -> 0.161; over_seg_guard NET PASS, no merge_rate
regression
  - new clips: GX010137 tolF1 0.448->0.685, GX010141 0.333->0.431 (both improve)


LENS = the 2 NEW clips (GX010137, GX010141). For each, report under baseline vs milestone:
num_gt, num_pred (windows), tolF1, merge_rate, split_rate, dend_abs_median, and the max window
duration.
Are they well-behaved or degenerate/blob-like (a single mega-window matching nothing like
mahadevpura_1)?
Do BOTH improve under the milestone? Is there any NEW failure mode the n=11 corpus didn't have
(e.g. a
clip the milestone makes WORSE, or a new over-merge/over-split pattern)? Adversarially: try to
find a
reason the new footage breaks the milestone story.

Return ONLY the structured verdict. Be precise with numbers.
```

2. assistant (2026-06-18T05:31:46.367Z)

I'll verify this claim by independently recomputing the numbers, focusing on the 2 new clips (GX010137, GX010141). Let me start by setting up and exploring the environment.

3. user (2026-06-18T05:31:48.411Z)

```
f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)
---MANIFEST---
Badminton_BXH_2.rallies.csv
Badminton_BXH_2_golden_features.json
GX010125_highlights.mp4
GX010125_highlights_min5.mp4
GX010125_run
GX010128.rallies.csv
GX010128_golden_features.json
GX010137.rallies.csv
GX010137_golden_features.json
GX010141.rallies.csv
GX010141_golden_features.json
GX030094.rallies.csv
GX030094_golden_features.json
KushagraAviHighlight-Trailor.mp4
MahadevpuraSingles_highlights.mp4
REAL_FOOTAGE_VALIDATION_REPORT.md
ab_experiment_n6.json
adarsh_avi_full_vs_human.csv
adarsh_avi_singles_golden_features.json
adarsh_avi_vs_human.csv
adarsh_baseline_gated.csv
adarsh_baseline_segments.csv
adarsh_gemini_fullvideo.csv
adarsh_gemini_refined.csv
adarsh_gemini_smoke.csv
adarsh_min5s_segments.csv
adarsh_tuned_segments.csv
badminton_indexer.db
chunks
e2e_reel.mp4
```

4. assistant (2026-06-18T05:31:51.073Z)

```
The worktree output is empty as warned. Let me write my verification script using the absolute
output path.
```

5. user (2026-06-18T05:31:53.984Z)

```
analyze_motion.py
clean_state.py
diagnose_rally.py
gemini_video_debugger.py
list_models.py
r1_evidence.py
test_ai_segmenter.py
test_thresholds.py
---API CHECK---
imports ok
```

6. assistant (2026-06-18T05:31:55.901Z)

```
Imports work. Now let me write the verification script.
```

7. assistant (2026-06-18T05:32:09.550Z)

```
import os, json
from dataclasses import replace
from backend.eval import rally_seg_eval as rse, eval_stats
from backend.eval.windowing import SERVED
from backend.eval.tolerance_metrics import over_seg_guard

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"

golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]
names = [g["name"] for g in golden]
print("N golden =", len(golden))
print("names =", sorted(names))

baseline  = SERVED
cap12     = replace(SERVED, max_window_duration=12.0)
cap6      = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)

def rows(p):
    return {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}

base_rows  = rows(baseline)
cap12_rows = rows(cap12)
cap6_rows  = rows(cap6)
mile_rows  = rows(milestone)

# ---- paired sign-tests on tolF1 ----
def paired(a_rows, b_rows, key="tol_f1"):
    a = [a_rows[n][key] for n in names]
    b = [b_rows[n][key] for n in names]
    return eval_stats.paired_delta_ci(a, b)

print("\n=== PAIRED tolF1 DELTAS (n=%d) ===" % len(names))
print("baseline->milestone:", json.dumps(paired(base_rows, mile_rows), indent=None))
print("cap12->cap6:        ", json.dumps(paired(cap12_rows, cap6_rows), indent=None))
print("cap6->+R4(milestone):", json.dumps(paired(cap6_rows, mile_rows), indent=None))
```

find the ONE loss in cap6->milestone

```
print("\n=== cap6->milestone per-clip tolF1 (find the loss) ===")
for n in sorted(names):
    d = mile_rows[n]["tol_f1"] - cap6_rows[n]["tol_f1"]
    flag = "  <-- LOSS" if d < -1e-9 else ("  (tie)" if abs(d) < 1e-9 else "")
    print(f"  {n:28s} cap6={cap6_rows[n]['tol_f1']:.4f} mile={mile_rows[n]['tol_f1']:.4f}
d={d:+.4f}{flag}")
```

---- merge_rate baseline->milestone ----

```
import statistics
mr_base = statistics.mean(base_rows[n]["merge_rate"] for n in names)
mr_mile = statistics.mean(mile_rows[n]["merge_rate"] for n in names)
print(f"\nmean merge_rate baseline={mr_base:.4f} -> milestone={mr_mile:.4f}")
```

---- over-seg guard ----

```
guard = over_seg_guard(list(base_rows.values()), list(mile_rows.values()))
print("\n=== over_seg_guard baseline->milestone ===")
print(json.dumps(guard, indent=2, default=str))
```

---- LENS: the 2 new clips ----

```
print("\n=== LENS: NEW CLIPS ===")
def maxwin(g, pred_cfg):
    res = rse.evaluate([g], pred_cfg)
    r = res["rows"][0]
    # try to get predicted windows durations
    return r

for clip in ["GX010137", "GX010141"]:
    if clip not in base_rows:
        print(f"  {clip}: NOT IN CORPUS")
        continue
    for tag, rr in [("baseline", base_rows), ("milestone", mile_rows)]:
        r = rr[clip]
        keys = ["num_gt", "num_pred", "tol_f1", "f1", "merge_rate", "split_rate",
                "dend_abs_median", "merge_count", "split_count"]
        vals = {k: r.get(k) for k in keys}
        print(f"  {clip} [{tag}]: {vals}")
    print()
```

8. user (2026-06-18T05:32:09.563Z)

```
File created successfully at: C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13/scratch/lens_new_clips.py (file state is current in your
context ? no need to Read it back)
```

9. user (2026-06-18T05:32:27.078Z)

```
N golden = 13
names = ['Badminton_BXH_2', 'GX010128', 'GX010137', 'GX010141', 'GX030094',
'adarsh_avi_singles', 'gbaaddy', 'kushagra_singles', 'largetest_doubles', 'mahadevpura_1',
'mahadevpura_2', 'mahadevpura_singles', 'testlarge_short']

=== PAIRED tolF1 DELTAS (n=13) ===
baseline->milestone: {"mean_delta": 0.2222020829574, "ci_low": 0.1521748189840441, "ci_high":
0.28586775911067197, "n": 13.0, "ci_excludes_zero": true, "sign_test": {"wins": 12.0, "losses":
0.0, "ties": 1.0, "n_effective": 12.0, "p_value": 0.00048828125}}
cap12->cap6:         {"mean_delta": 0.10988694701449651, "ci_low": 0.0702621335426765,
"ci_high": 0.15261417368181246, "n": 13.0, "ci_excludes_zero": true, "sign_test": {"wins": 12.0,
"losses": 0.0, "ties": 1.0, "n_effective": 12.0, "p_value": 0.00048828125}}
cap6->+R4(milestone): {"mean_delta": 0.03969788845339993, "ci_low": 0.014483842422685857,
"ci_high": 0.06513158979036085, "n": 13.0, "ci_excludes_zero": true, "sign_test": {"wins": 9.0,
"losses": 1.0, "ties": 3.0, "n_effective": 10.0, "p_value": 0.021484375}}
```

```
=== cap6->milestone per-clip tolF1 (find the loss) ===
  Badminton_BXH_2              cap6=0.2273 mile=0.3279 d=+0.1006
  GX010128                     cap6=0.3472 mile=0.4662 d=+0.1189
  GX010137                     cap6=0.7273 mile=0.6849 d=-0.0423  <-- LOSS
  GX010141                     cap6=0.3929 mile=0.4314 d=+0.0385
  GX030094                     cap6=0.3538 mile=0.4194 d=+0.0655
  adarsh_avi_singles           cap6=0.4957 mile=0.5520 d=+0.0564
  gbaaddy                      cap6=0.1854 mile=0.2778 d=+0.0923
  kushagra_singles             cap6=0.0741 mile=0.0779 d=+0.0038
  largetest_doubles            cap6=0.5645 mile=0.5833 d=+0.0188
  mahadevpura_1                cap6=0.0000 mile=0.0000 d=+0.0000  (tie)
  mahadevpura_2                cap6=0.1220 mile=0.1220 d=+0.0000  (tie)
  mahadevpura_singles          cap6=0.4667 mile=0.5301 d=+0.0635
  testlarge_short              cap6=0.2000 mile=0.2000 d=+0.0000  (tie)

mean merge_rate baseline=0.6513 -> milestone=0.1610

=== over_seg_guard baseline->milestone ===
{
  "passes": true,
  "strict_passes": false,
  "net_cost_ok": true,
  "no_merge_rate_regress": true,
  "base_net": 1.3025976530825853,
  "cand_net": 0.4289172588246174,
  "net_delta": -0.873680394257968,
  "n": 13,
  "use_rates": true,
  "weight_merge": 2.0,
  "weight_split": 1.0,
  "new_merges": [
    {
      "name": "kushagra_singles",
      "base": 2.0,
      "cand": 3.0
    },
    {
      "name": "mahadevpura_2",
      "base": 3.0,
      "cand": 9.0
    },
    {
      "name": "mahadevpura_1",
      "base": 1.0,
      "cand": 2.0
    },
    {
      "name": "GX010141",
      "base": 5.0,
      "cand": 6.0
    }
  ],
  "new_splits": [
    {
      "name": "adarsh_avi_singles",
      "base": 0.0,
      "cand": 13.0
    },
    {
      "name": "kushagra_singles",
      "base": 0.0,
      "cand": 6.0
    },
    {
      "name": "largetest_doubles",
```

```
      "base": 0.0,
      "cand": 5.0
    },
    {
      "name": "gbaaddy",
      "base": 0.0,
      "cand": 8.0
    },
    {
      "name": "testlarge_short",
      "base": 0.0,
      "cand": 1.0
    },
    {
      "name": "mahadevpura_singles",
      "base": 0.0,
      "cand": 7.0
    },
    {
      "name": "mahadevpura_2",
      "base": 0.0,
      "cand": 4.0
    },
    {
      "name": "GX010128",
      "base": 0.0,
      "cand": 2.0
    },
    {
      "name": "GX030094",
      "base": 0.0,
      "cand": 6.0
    },
    {
      "name": "Badminton_BXH_2",
      "base": 0.0,
      "cand": 4.0
    },
    {
      "name": "GX010137",
      "base": 0.0,
      "cand": 1.0
    }
  ],
  "merge_rate_regress": []
}

=== LENS: NEW CLIPS ===
  GX010137 [baseline]: {'num_gt': 34, 'num_pred': 33, 'tol_f1': 0.4477611940298507, 'f1':
0.5970149253731343, 'merge_rate': 0.17647058823529413, 'split_rate': 0.0, 'dend_abs_median':
1.2181773440106838, 'merge_count': 3, 'split_count': 0}
  GX010137 [milestone]: {'num_gt': 34, 'num_pred': 39, 'tol_f1': 0.6849315068493151, 'f1':
0.8219178082191781, 'merge_rate': 0.0, 'split_rate': 0.029411764705882353, 'dend_abs_median':
0.43861027694362065, 'merge_count': 0, 'split_count': 1}

  GX010141 [baseline]: {'num_gt': 29, 'num_pred': 19, 'tol_f1': 0.3333333333333333, 'f1':
0.3333333333333333, 'merge_rate': 0.6551724137931034, 'split_rate': 0.0, 'dend_abs_median':
5.431222889556238, 'merge_count': 5, 'split_count': 0}
  GX010141 [milestone]: {'num_gt': 29, 'num_pred': 22, 'tol_f1': 0.4313725490196078, 'f1':
0.5098039215686274, 'merge_rate': 0.4482758620689655, 'split_rate': 0.0, 'dend_abs_median':
0.6764572906239579, 'merge_count': 6, 'split_count': 0}
```


10. assistant (2026-06-18T05:32:31.746Z)

All my numbers match the orchestrator's. Now let me dig into the lens question: I found

something noteworthy. The orchestrator claimed "GX010137 tolF1 0.448->0.685 (both improve)" but my per-clip cap6->milestone breakdown shows GX010137 is actually the ONE LOSS in the cap6->milestone step (0.727->0.685). It still improves baseline->milestone. Let me investigate the max window durations and the full baseline->cap6 trajectory for the new clips to characterize them adversarially.

11. assistant (2026-06-18T05:32:37.888Z)

```
import os
from dataclasses import replace
from backend.eval import rally_seg_eval as rse
from backend.eval.windowing import SERVED

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"
golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]
gmap = {g["name"]: g for g in golden}

baseline  = SERVED
cap12     = replace(SERVED, max_window_duration=12.0)
cap6      = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
```

Inspect how the pipeline produces windows. Find the function that turns a golden+cfg

```
print("evaluate row keys:", list(rse.evaluate([gmap['GX010137']], baseline)["rows"][0].keys()))
```

Look for a windows builder in rse

```
funcs = [n for n in dir(rse) if not n.startswith("_")]
print("rse public:", funcs)
```

12. user (2026-06-18T05:32:37.898Z)

```
File created successfully at: C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13/scratch/lens_windows.py (file state is current in your
context ? no need to Read it back)
```

13. user (2026-06-18T05:32:43.473Z)

```
evaluate row keys: ['f1', 'precision', 'recall', 'mean_iou', 'mean_start_error',
'mean_end_error', 'f1@0.1', 'f1@0.25', 'f1@0.5', 'segment_count_ratio', 'merge_rate',
'split_rate', 'merges', 'splits', 'missed', 'spurious', 'num_pred', 'num_gt', 'tau_start',
'tau_end', 'tol_f1', 'tol_precision', 'tol_recall', 'tol_matched', 'dstart_abs_median',
'dend_abs_median', 'split_count', 'merge_count', 'name']
rse public: ['Any', 'DEFAULT_FEATURES_DIR', 'DEFAULT_IOU', 'DEFAULT_MANIFEST', 'Dict',
'Interval', 'List', 'Optional', 'PRESETS', 'SERVED', 'WindowingPreset', 'aggregate',
'annotations', 'argparse', 'compare', 'eval_stats', 'evaluate', 'format_report', 'json',
'load_golden', 'main', 'metrics', 'os', 'score_intervals', 'segmentation_metrics', 'windowing',
'windows_for']
```

14. assistant (2026-06-18T05:32:46.606Z)

There's a `windows_for` function. Let me get the actual predicted windows and their max durations for the two new clips under baseline and milestone.

15. assistant (2026-06-18T05:32:54.703Z)

```
import os, inspect
from dataclasses import replace
from backend.eval import rally_seg_eval as rse
from backend.eval.windowing import SERVED

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"
```

```
golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]
gmap = {g["name"]: g for g in golden}

print("windows_for sig:", inspect.signature(rse.windows_for))

baseline  = SERVED
cap6      = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)

def wins(g, cfg):
    w = rse.windows_for(g, cfg)
    return w

for clip in ["GX010137", "GX010141", "mahadevpura_1"]:
    g = gmap[clip]
    gts = g["gts"]
    print(f"\n===== {clip} =====")
    print(f"  num_gt={len(gts)}  gt durations: min={min(b-a for a,b in gts):.2f} max={max(b-a
for a,b in gts):.2f} "
          f"mean={sum(b-a for a,b in gts)/len(gts):.2f}
span=[{gts[0][0]:.1f},{gts[-1][1]:.1f}]")
    for tag, cfg in [("baseline", baseline), ("cap6", cap6), ("milestone", milestone)]:
        w = wins(g, cfg)
        durs = [b-a for a,b in w]
        if durs:
            print(f"  [{tag:9s}] n={len(w):3d}  durs: min={min(durs):.2f} max={max(durs):.2f} "
                  f"mean={sum(durs)/len(durs):.2f}  total_cover={sum(durs):.1f}s")
        else:
            print(f"  [{tag:9s}] n=0 (no windows!)")
```

16. user (2026-06-18T05:32:54.721Z)

```
File created successfully at: C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13/scratch/lens_windows2.py (file state is current in your
context ? no need to Read it back)
```

17. user (2026-06-18T05:33:01.464Z)

```
windows_for sig: (video: 'Dict[str, Any]', preset: 'WindowingPreset', min_crossings: 'int' = 0)
-> 'List[Interval]'

===== GX010137 =====
  num_gt=34  gt durations: min=1.74 max=11.75 mean=5.16  span=[5.2,440.0]
  [baseline ] n= 33  durs: min=2.15 max=33.03 mean=8.47  total_cover=279.6s
  [cap6     ] n= 43  durs: min=2.14 max=17.07 mean=4.96  total_cover=213.1s
  [milestone] n= 39  durs: min=2.04 max=15.02 mean=4.82  total_cover=188.1s

===== GX010141 =====
  num_gt=29  gt durations: min=2.76 max=10.52 mean=5.02  span=[9.2,371.5]
  [baseline ] n= 19  durs: min=2.34 max=101.08 mean=15.24  total_cover=289.6s
  [cap6     ] n= 27  durs: min=2.17 max=36.32 mean=9.99  total_cover=269.8s
  [milestone] n= 22  durs: min=2.17 max=30.38 mean=10.22  total_cover=224.9s

===== mahadevpura_1 =====
  num_gt=10  gt durations: min=3.00 max=13.25 mean=6.35  span=[18.2,146.2]
  [baseline ] n=  1  durs: min=174.27 max=174.27 mean=174.27  total_cover=174.3s
  [cap6     ] n=  2  durs: min=71.91 max=101.89 mean=86.90  total_cover=173.8s
  [milestone] n=  2  durs: min=60.38 max=65.78 mean=63.08  total_cover=126.2s
```

18. assistant (2026-06-18T05:33:08.593Z)

This is very informative. Now I have a clear picture. Note that `windows_for` shows different

`num_pred` than the evaluate rows (e.g. GX010137 cap6 shows n=43 here vs the cap window cap was applied ? but the cap is `max_window_duration=6` yet max dur is 17.07s). Let me understand: the cap doesn't hard-truncate to 6s; the max durations exceed 6 even under cap6. Let me check the windowing semantics to characterize correctly, and confirm whether the milestone caps windows. But the key lens findings are already clear. Let me note that GX010137 max window is 15-17s (not a single mega-window), GX010141 is 30s under milestone (down from 101s baseline). Neither is a degenerate single-mega-window like mahadevpura_1 (1 window of 174s).

Let me verify the cap semantics quickly to report `max window duration` accurately, and double-check the GX010137 cap6->milestone regression detail.

19. user (2026-06-18T05:33:10.009Z)

```
backend/eval/windowing.py:63:    max_window_duration: float = 0.0
backend/eval/windowing.py:64:    #: When True, ``max_window_duration`` is a HARD cap ? an over-
long window with no internal
backend/eval/windowing.py:74:    #: longer than ``blob_factor`` × ``max_window_duration`` (the
gap-less, R4-survived blob ?
backend/eval/windowing.py:83:        bounded-windowing knob ``max_window_duration`` is passed
separately by ``build_windows``.)"""
backend/eval/windowing.py:92:            "max_window_duration": self.max_window_duration,
backend/eval/windowing.py:120:        max_window_duration=float(idx.max_window_duration),  # W1
bounded-windowing (default 0=OFF)
backend/eval/windowing.py:162:        max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
backend/eval/windowing.py:199:        max_window_duration=preset.max_window_duration,
```

20. assistant (2026-06-18T05:33:19.714Z)

The `max_window_duration` is a soft cap (splits at internal gaps/inactivity); windows can exceed it if there's no internal gap to split at ? that's exactly why R5 blob-fallback exists for the gap-less mega-blob (mahadevpura_1). The key point: for both new clips, the milestone produces many windows (39, 22) with bounded max durations (15s, 30s), not a single mega-window. They are well-behaved relative to the degenerate mahadevpura_1 (single 174s / two 60-66s windows matching almost nothing, tolF1=0.0).

I have everything I need. Let me confirm the final assessment of the orchestrator's specific claims about the new clips and the milestone net result. My numbers fully confirm the headline. The one nuance: the orchestrator said "both improve" baseline->milestone (TRUE: GX010137 0.448->0.685, GX010141 0.333->0.431), but GX010137 is the single cap6->milestone LOSS (0.727->0.685) ? R4 mildly over-splits it (split_count 0->1, dend improves 1.22->0.44 though). This is consistent with the orchestrator's note "(ONE loss ? find it)". The loss is a NEW clip, but it's a within-step regression, not a baseline regression, and the over-seg guard still NET-PASSES.

21. user (2026-06-18T05:33:40.533Z)

Structured output provided successfully

22. assistant (2026-06-18T05:33:49.533Z)

Verdict: **confirms**. I independently recomputed every number against the n=13 corpus on worktree HEAD f5f339a using the public API, and they match the orchestrator's to 4 decimal places.

Key findings on the LENS (the 2 new clips):
- **Both improve baseline->milestone**: GX010137 0.448->0.685, GX010141 0.333->0.431. Confirmed.
- **Neither is degenerate/blob-like.** GX010137 milestone = 39 windows (max ~15s), GX010141 = 22 windows (max ~30s). Contrast with the genuine blob mahadevpura_1 (single 174s window, tolF1=0.0, unchanged and legacy).
- **The single cap6->milestone tolF1 loss IS a new clip**: GX010137 (0.7273->0.6849, ?-0.0423). R4 mildly over-splits it (split_count 0->1) but sharply improves its end precision (dend 1.218->0.439, merge_rate 0.176->0.0). It's a within-step dip, not a baseline regression, and doesn't threaten the promotion bar.

- **No new failure mode.** GX010141 is the worst-behaved new clip post-milestone (merge_rate still 0.448) but it improves and its merge_rate drops ? a future over-merge target, not a regression.

The milestone still stands: significant baseline->milestone tolF1 win (12/0/1, p=0.0005) + over_seg_guard net-pass (net cost 1.30->0.43, no merge_rate regression). One honest nuance the headline omitted: strict_passes=false (R4 broadens splits across 11 clips), but the net-cost + no-merge-rate-regress promotion bar is the bar, and it clears.